

What Neural Networks Memorize and Why: Discovering the Long Tail via Influence Estimation

Vitaly Feldman * [†]
Apple

Chiyuan Zhang*
Google Research, Brain Team

Abstract

Deep learning algorithms are well-known to have a propensity for fitting the training data very well and often fit even outliers and mislabeled data points. Such fitting requires memorization of training data labels, a phenomenon that has attracted significant research interest but has not been given a compelling explanation so far. A recent work of Feldman [Fel19] proposes a theoretical explanation for this phenomenon based on a combination of two insights. First, natural image and data distributions are (informally) known to be long-tailed, that is have a significant fraction of rare and atypical examples. Second, in a simple theoretical model such memorization is necessary for achieving close-to-optimal generalization error when the data distribution is long-tailed. However, no direct empirical evidence for this explanation or even an approach for obtaining such evidence were given.

In this work we design experiments to test the key ideas in this theory. The experiments require estimation of the influence of each training example on the accuracy at each test example as well as memorization values of training examples. Estimating these quantities directly is computationally prohibitive but we show that closely-related *subsampled* influence and memorization values can be estimated much more efficiently. Our experiments demonstrate the significant benefits of memorization for generalization on several standard benchmarks. They also provide quantitative and visually compelling evidence for the theory put forth in [Fel19].

1 Introduction

Perhaps the most captivating aspect of deep learning algorithms is their ability to generalize to unseen data. The models used in deep learning are typically overparameterized, making it easy to perfectly fit the training dataset without any generalization. In fact, the standard training algorithms do fit the training data very well, typically achieving 95-100% accuracy, even when the accuracy on the test dataset is much more modest. In particular, they usually fit obvious outliers (such as images with no discernible features of their class) and mislabeled examples. The only way for a training algorithm to fit an example whose label cannot be predicted based on the rest of the dataset is to memorize¹ the label. Further, it is now well-known that standard deep learning algorithms achieve high training accuracy even on large and randomly labeled datasets [ZBHRV17].

This propensity for label memorization is not explained by the standard approach to understanding of generalization. At a high level, the standard approach upper-bounds the generalization error by the sum of an upper bound on the generalization gap controlled by a model complexity (or stability) parameter and the empirical error. Fitting of outliers and mislabeled examples does not improve the generalization error. Therefore, to avoid “overfitting”, the balance between the complexity parameter and the empirical error is supposed to be tuned in a way that prevents label memorization. Memorization is also generally thought of (and taught in ML courses) as being the opposite of generalization.

This disconnect between the classical theory and modern practice was highlighted in the work of Zhang et al. [ZBHRV17] and generated a large wave of research interest in the topic of generalization for deep learning. The bulk of this research focuses on finding new ways to control the generalization gap or showing that training algorithms

*Equal contribution.

[†]Part of this work done while the author was at Google Research, Brain Team.

¹This notion of label memorization is defined rigorously in eq. (1) (Sec. 1.1).

induce a form of implicit regularization. These results have led to tighter theoretical bounds and, in some cases, bounds that show correlation with the actual generalization gap (see [NBMS17; NK19; JNMKB20] for analyses of numerous measures). Yet, fundamentally, these works still follow the same classical approach to generalization that cannot explain memorization. Another line of research focuses on the generalization error of algorithms that fit the data perfectly (referred to as interpolating) [WOBM17; BRT18; BHM18; LR18; BMM18; RZ19; BLLT19; BHX19; HMRT19; MVS19]. At a high level, these works show that under certain conditions interpolating algorithms achieve (asymptotically) optimal generalization error. However, under the same conditions there also exist standard non-interpolating algorithms that achieve the optimal generalization error (e.g. via appropriate regularization). Thus these works do not explain why interpolating algorithms are used in the first place.

A recent work of Feldman [Fel19] proposes a new explanation for the benefits of memorization. The explanation suggests that memorization is necessary for achieving close-to-optimal generalization error when the data distribution is long-tailed, namely, rare and atypical instances make up a significant fraction of the data distribution. Moreover, in such distributions useful examples from the “long tail” (in the sense that memorizing them improves the generalization error) can be statistically indistinguishable from the useless one, such as outliers and mislabeled ones. This makes memorization of useless examples (and the resulting large generalization gap) necessary for achieving close-to-optimal generalization error. We will refer to this explanation as the *long tail theory*.

In [Fel19] the need for memorization and statistical indistinguishability of useful from useless examples are theoretically demonstrated using an abstract model. In this model the data distribution is a mixture of subpopulations and the frequencies of those subpopulations are chosen from a long-tailed prior distribution. Subpopulation are presumed to be distinct enough from each other that a learning algorithm cannot achieve high accuracy on a subpopulation without observing any representatives from it. The results in [Fel19] quantify the cost of not memorizing in terms of the prior distribution and the size of the dataset n . They also show that the cost is significant for the prototypical long-tailed distributions (such as the Zipf distribution) when the number of samples is smaller than the number of subpopulations.

While it has been recognized in many contexts that modern datasets are long-tailed [ZAR14; VHP17; BS19], it is unclear whether this has any relationship to memorization by modern deep learning algorithms and (if so) how significant is the effect. The theoretical explanation in [Fel19] is based on a generative prior distribution and therefore cannot be directly verified. This leads to the question of how the long tail theory can be tested empirically.

1.1 Overview

In this work we develop approaches for empirically validating the long tail theory. The starting point for such validation is examining which training examples are memorized and what is the utility of all the memorized examples as a whole. To make it more concrete we recall the definition of label memorization from [Fel19]. For a training algorithm $\mathcal{A}$ operating on a dataset $S = ((x_1, y_1), \dots, (x_n, y_n))$ the amount of label memorization by $\mathcal{A}$ on example $(x_i, y_i) \in S$ is defined as

$$\text{mem}(\mathcal{A}, S, i) := \Pr_{h \leftarrow \mathcal{A}(S)}[h(x_i) = y_i] - \Pr_{h \leftarrow \mathcal{A}(S \setminus i)}[h(x_i) = y_i], \quad (1)$$

where $S \setminus i$ denotes the dataset S with (x_i, y_i) removed and probability is taken over the randomness of the algorithm $\mathcal{A}$ (such as random initialization). This definition captures and quantifies the intuition that an algorithm memorizes the label y_i if its prediction at x_i based on the rest of the dataset changes significantly once (x_i, y_i) is added to the dataset.

The primary issue with this definition is that estimating memorization values with standard deviation of σ requires running $\mathcal{A}(S \setminus i)$ on the order of $1/\sigma^2$ times for every example. As a result, this approach requires $\Omega(n/\sigma^2)$ training runs which translates into millions of training runs needed to achieve $\sigma < 0.1$ on a dataset with $n = 50,000$ examples. We bypass this problem by proposing a closely-related estimator that looks at the expected memorization of the label of x_i on a random subset of S that includes $m \leq n$ of examples. It can be seen as $\text{mem}(\mathcal{A}, S, i)$ smoothed by the random subsampling of the dataset and is also related to the Shapley value of example i for accuracy on itself. Most importantly, for m bounded away from n and 1 this memorization value can be estimated with standard deviation σ for every i at the same time using just $O(1/\sigma^2)$ training runs.

We compute memorization value estimates on the MNIST, CIFAR-100 and ImageNet datasets and then estimate the marginal effect of memorized examples on the test accuracy by removing those examples from the training dataset.

We find that, aside from the MNIST dataset,² a significant fraction of examples have large memorization estimates. The marginal utility of the memorized examples is also significant, in fact higher than a random set of examples of the same size. For example, on the ImageNet $\approx 32\%$ of examples have memorization estimates ≥ 0.3 and their marginal utility is $\approx 3.4\%$ (vs. $\approx 2.6\%$ for a random subset of 32% of examples). In addition, by visually examining the memorization estimates, we see that examples with high memorization scores are a mixture of atypical examples and outliers/mislabeled examples (whereas examples with low memorization estimates are much more typical). All of these findings are consistent with the long tail theory.

A more important prediction of the theory is that memorization is necessary since each memorized representative of a rare subpopulation significantly increases the prediction accuracy on its subpopulation. We observe that this prediction implies that there should exist a substantial number of memorized training examples each of which significantly increases the accuracy on an example in the test set. Further, of the test examples that are influenced significantly, most are influenced significantly only by a single training example. The uniqueness is important since, according to the theoretical results, such unique representatives of a subpopulation are the ones that are hard to distinguish from outliers and mislabeled examples (and thus memorizing them requires also memorizing useless examples).

To find such high-influence pairs of examples we need to estimate the influence of each training example (x_i, y_i) on the accuracy of the algorithm $\mathcal{A}$ at each test example (x'_j, y'_j) :

$$\text{infl}(\mathcal{A}, S, i, j) := \Pr_{h \leftarrow \mathcal{A}(S)}[h(x'_j) = y'_j] - \Pr_{h \leftarrow \mathcal{A}(S \setminus i)}[h(x'_j) = y'_j]. \quad (2)$$

As with memorization values, estimating the influence values for all pairs of examples is clearly not computationally feasible. A famous proxy for the classical leave-one-one influence is the influence function [CW82]. Computing this function for deep neural networks has been studied recently by Koh and Liang [KL17] who proposed a method based on assumptions of first and second order optimality. Alternative proxies for measuring influence have been studied in [YKYR18; PLSK20].

We propose and use a new estimator for influence based on the same subsampling as our memorization value estimator. Its primary advantages are that it is a natural smoothed version of the influence value itself and, as we demonstrate visually, provides reliable and relatively easy to interpret estimates. We then locate all train-test pairs of examples from the same class (x_i, y_i) and (x'_j, y'_j) such that our estimate of the memorization value of (x_i, y_i) is sufficiently large (we chose 0.25 as the threshold) and our estimate of the influence of (x_i, y_i) on the accuracy at (x'_j, y'_j) is significant (we chose 0.15 as the threshold). See Sec. 2 for the details of the estimator and the justification of the threshold choice. Overall, we found a substantial number of such pairs in the CIFAR-100 and the ImageNet. For example we found 1641 pairs satisfying these criteria in the ImageNet. In these pairs there are 1462 different test examples (comprising 2.92% of the test set) of which 1298 are influenced significantly by only a single training example.

These quantitative results of our experiments clearly support the key ideas of the long tail theory. To further investigate the findings, we visually inspect the high-influence pairs of examples that were found by our methods. This inspection shows that, in most cases, the pairs have an easy-to-interpret visual similarity and provide, we believe, the most compelling evidence for the long tail theory.

In addition to our main experiments, we investigate several natural related questions (albeit only on CIFAR-100). In the first set of experiments we look at how much the results of our experiments depend on the choice of the architecture. We find that, while the architecture definitely plays a role (as it does for the accuracy) there are strong correlations between sets of memorized examples and high-influence pairs for different architectures. These experiments also give a sense of how much our results are affected by the randomness in the estimation and training processes and the resulting selection bias.

Finally, as our influence and memorization estimators are still very computationally intensive, we consider a faster way to compute these values. Specifically, instead of training the entire network on each random subset of S , we train only the last layer over the representation given by the penultimate layer of the network trained once on the entire dataset. The resulting estimator is much more computationally efficient but it fails completely at detecting memorized examples and gives much worse influence estimates. In addition to being a potentially useful negative result, this

²We include the MNIST dataset primarily as a comparison point, to show that memorization plays a much smaller role when the variability of the data is low (corresponding to a low number of subpopulations in a mixture model) and the number of examples per class is high.